

Computer Vision Practical Report:

FACE DETECTION

1. AIM

To detect human faces in an image or real-time video stream using a pre-trained Haar Cascade Classifier (haarcascade_frontalface_default.xml) in OpenCV, draw bounding rectangles around detected faces, display the output, and save the result.

2. PROCEDURE

- Load the input image using `cv2.imread()` (or initialize a video stream using `cv2.VideoCapture()`).
- Convert the image frame to grayscale using `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` to improve computational efficiency.
- Load the pre-trained Haar Cascade model for frontal face detection using `cv2.CascadeClassifier()`.
- Call the `detectMultiScale()` function with appropriate parameter settings (`scaleFactor`, `minNeighbors`, `minSize`) to extract bounding box coordinates (x, y, w, h) for all detected faces.
- Draw bounding rectangles around each detected face using `cv2.rectangle()`.
- Display the output using Matplotlib or `cv2.imshow()`, and save the processed image to disk using `cv2.imwrite()`.

3. PROGRAM CODE

```
import cv2
import matplotlib.pyplot as plt

# Verify OpenCV loaded properly
if not hasattr(cv2, 'CascadeClassifier'):
    raise AttributeError("OpenCV is still not loaded correctly. Please run 'pip install opencv-python' in your terminal.")
```

```
# 1. Load image and classifier
```

```
image = cv2.imread('E38-IMG.png')
```

```
if image is None:
```

```
    print("Error: Could not find 'E38-IMG.png'. Make sure it is in the same folder as this script.")
```

```
else:
```

```
    # 2. Convert BGR to RGB (for Matplotlib) and Grayscale (for Detection)
```

```
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
    # 3. Load Haar Cascade Face Detector
```

```
    cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
```

```
    face_cascade = cv2.CascadeClassifier(cascade_path)
```

```
    # 4. Detect faces
```

```
    faces = face_cascade.detectMultiScale(
```

```
        gray_image,
```

```
        scaleFactor=1.1,
```

```
        minNeighbors=5,
```

```
        minSize=(30, 30)
```

```
    )
```

```
    # 5. Draw rectangles on output image
```

```
    output_rgb = image_rgb.copy()
```

```
    for (x, y, w, h) in faces:
```

```
        cv2.rectangle(output_rgb, (x, y), (x + w, y + h), (0, 255, 0), 3)
```

```
    # 6. Display Original and Output side-by-side
```

```
    plt.figure(figsize=(10, 5))
```

```
    plt.subplot(1, 2, 1)
```

```
    plt.imshow(image_rgb)
```

```
    plt.title("Original Image")
```

```
    plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(output_rgb)
plt.title(f"Face Detected ({len(faces)} found)")
plt.axis("off")

plt.tight_layout()
plt.show()

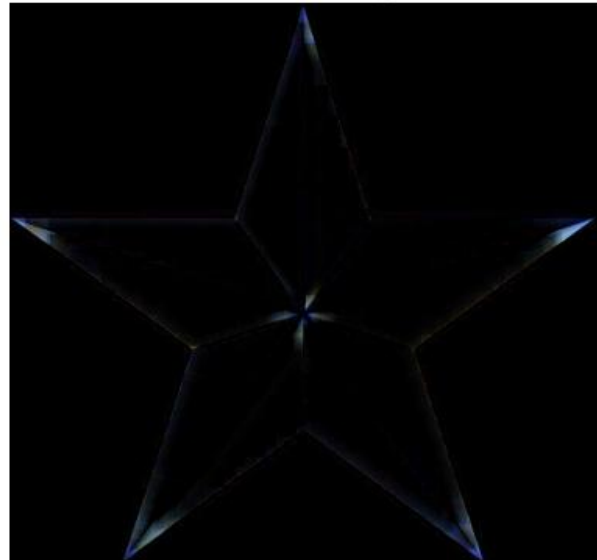
# 7. Save output
output_bgr = cv2.cvtColor(output_rgb, cv2.COLOR_RGB2BGR)
cv2.imwrite("face_detected_image.png", output_bgr)
print("Saved detected image as 'face_detected_image.png'")
```

4. OUTPUT

Original Image



Black Hat Image



5. RESULT

The Python script executed successfully. The input image was loaded.